

Estrazione di feature

Annalisa Franco
annalisa.franco@unibo.it

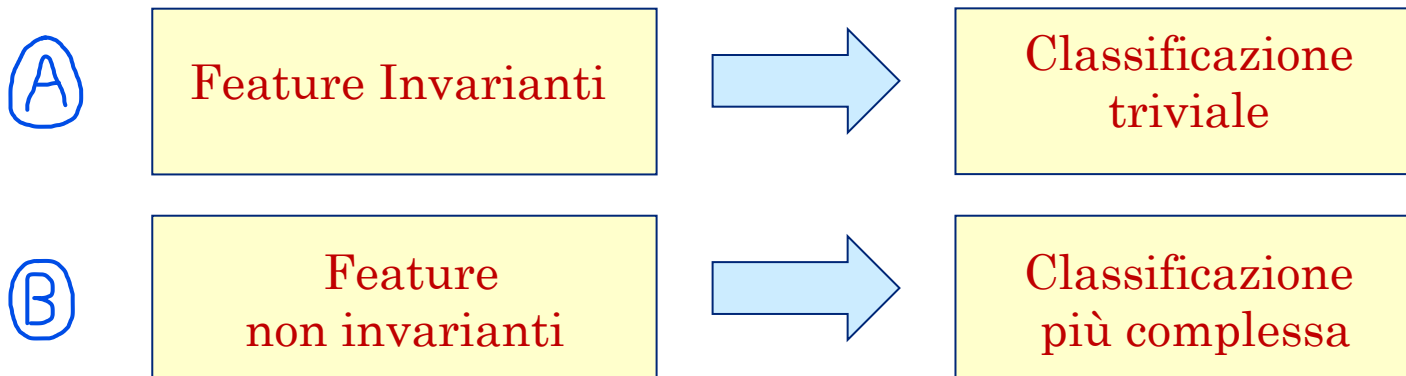
le differenze che possono esistere all'interno della stessa classe tra immagini diverse. Ci sono due tipi di differenze: Dello stesso oggetto in condizioni diverse (Rotazione, Scala/Dimensione, Posizione, Illuminazione) e Tra soggetti diversi della stessa categoria.

Feature Invarianti (1)

Con il termine **feature invarianti** si denotano caratteristiche estratte dai pattern che siano costanti (il più possibile) rispetto alle possibili variazioni intra-classe (ad esempio due semplici feature invarianti rispetto a trasformazioni affini sono **area** e **perimetro** di un oggetto).

Per alcune trasformazioni, certe feature sono invarianti ma per altre trasformazioni no

Uno dei maggiori dilemmi in applicazioni di PR è se l'**invarianza dei pattern** debba essere gestita ^(A) a livello di feature (attraverso la scelta e l'estrazione di feature invarianti) o ^(B) a livello di classificazione (attraverso la scelta e il progetto di metodi in grado di sopportare feature non invarianti). Problema: Dove gestire la variabilità intra-classe? Definire feature invarianti estratte dai pattern oppure gestire la variabilità intra-classe in base alla scelta di modelli di classificazione



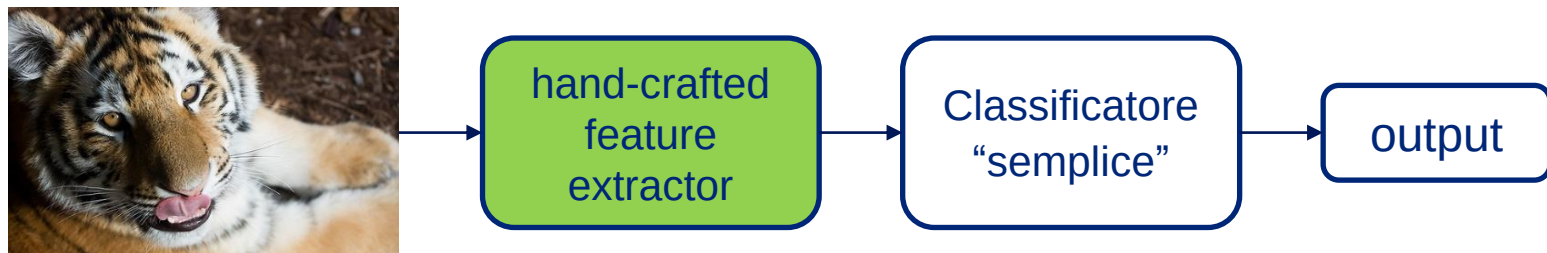
Feature Invarianti (2)

Nessuno dei due metodi è preferibile in assoluto, e molto spesso gli approcci scelti sono ibridi (che gestiscono l'invarianza a livello di feature solo per alcuni tipi di variabilità).

- Generalmente se l'invarianza è gestita a livello di feature, la classificazione è molto più veloce.
- D'altro canto, spesso la gestione dell'invarianza a livello di feature è più difficile, e i metodi che utilizzano classificatori robusti (e feature non invarianti) forniscono maggiore accuratezza e affidabilità.

Hand-crafted features vs. representation learning

- Hand-crafted features: rappresentazione numerica ottenuta attraverso processi di feature extraction definiti dal progettista allo scopo di codificare al meglio le caratteristiche salienti delle immagini (colore, forma, tessitura).
- Representation learning: apprendimento automatico di feature salienti attraverso tecniche di deep learning. Le reti ricevono in input pattern «grezzi» (es. immagini) e in modo autonomo apprendono la rilevanza delle feature a diversi livelli di astrazione.



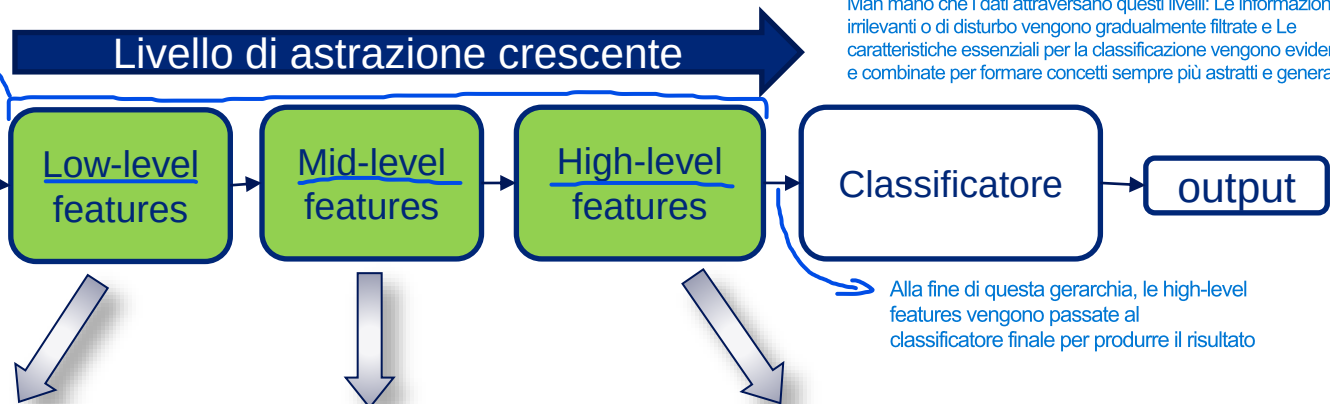
Representation Learning

- Le tecniche basate su deep learning cercano di apprendere rappresentazioni gerarchiche significative (features) automaticamente, attraverso un processo di apprendimento costituito da più livelli.
- Ogni livello opera una sorta di trasformazione non lineare di feature, i cui parametri possono essere appresi automaticamente.

Vengono scelte automaticamente, per ogni livello, le features più significative per la classificazione

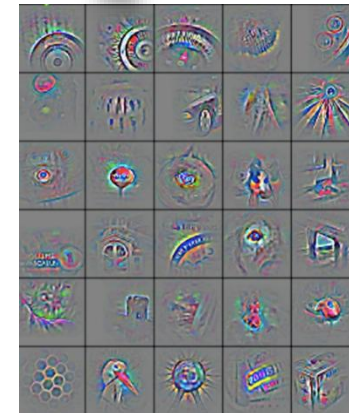
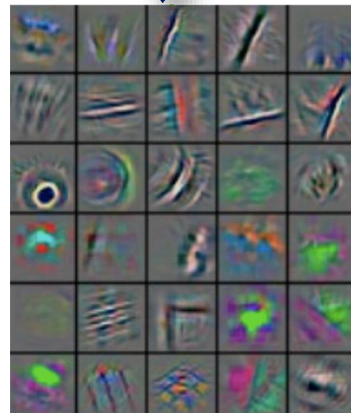
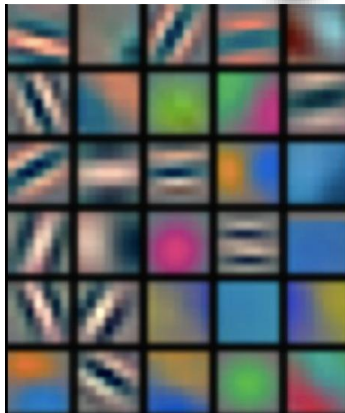
alle feature individuate dal livello precedente.

la rete impara a comprendere l'immagine costruendo la conoscenza a livelli di astrazione crescente.



Man mano che i dati attraversano questi livelli: Le informazioni irrilevanti o di disturbo vengono gradualmente filtrate e Le caratteristiche essenziali per la classificazione vengono evidenziate e combinate per formare concetti sempre più astratti e generali.

Alla fine di questa gerarchia, le high-level features vengono passate al classificatore finale per produrre il risultato



Feature visualization of convolutional net trained on ImageNet (Zeiler and Fergus, 2013)

Quale approccio scegliere?

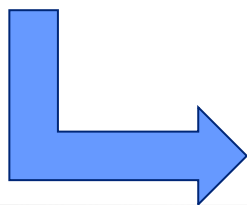
- Non esiste una tecnica universalmente migliore dell'altra; applicazioni diverse richiedono tecniche diverse.
- Le tecniche hand-crafted:
 - Possono essere applicate anche in domini in cui sono disponibili quantità di dati limitate;
 - Sono più facilmente «interpretabili»;
 - Non è sempre facile esplicitare il contenuto informativo di un'immagine attraverso feature di basso livello (specifiche per il problema in oggetto).
- Le tecniche basate su deep-learning:
 - Hanno un carattere più generale, adattandosi talvolta a problemi differenti;
 - L'addestramento di una rete richiede tipicamente grandi quantità di dati (possibile spesso usare reti pre-addestrate);
 - Difficilmente interpretabili, necessità di raggiungere una maggiore comprensione dei processi che guidano l'apprendimento.

i tre principali campi di applicazione in cui vengono utilizzate le feature invarianti nel campo della Pattern Recognition e della Computer Vision

Feature invariant: applicazioni (1)

Rappresenta la ricerca di immagini basata sul contenuto visivo (come la ricerca per immagini di Google).

Ricerche di similarità

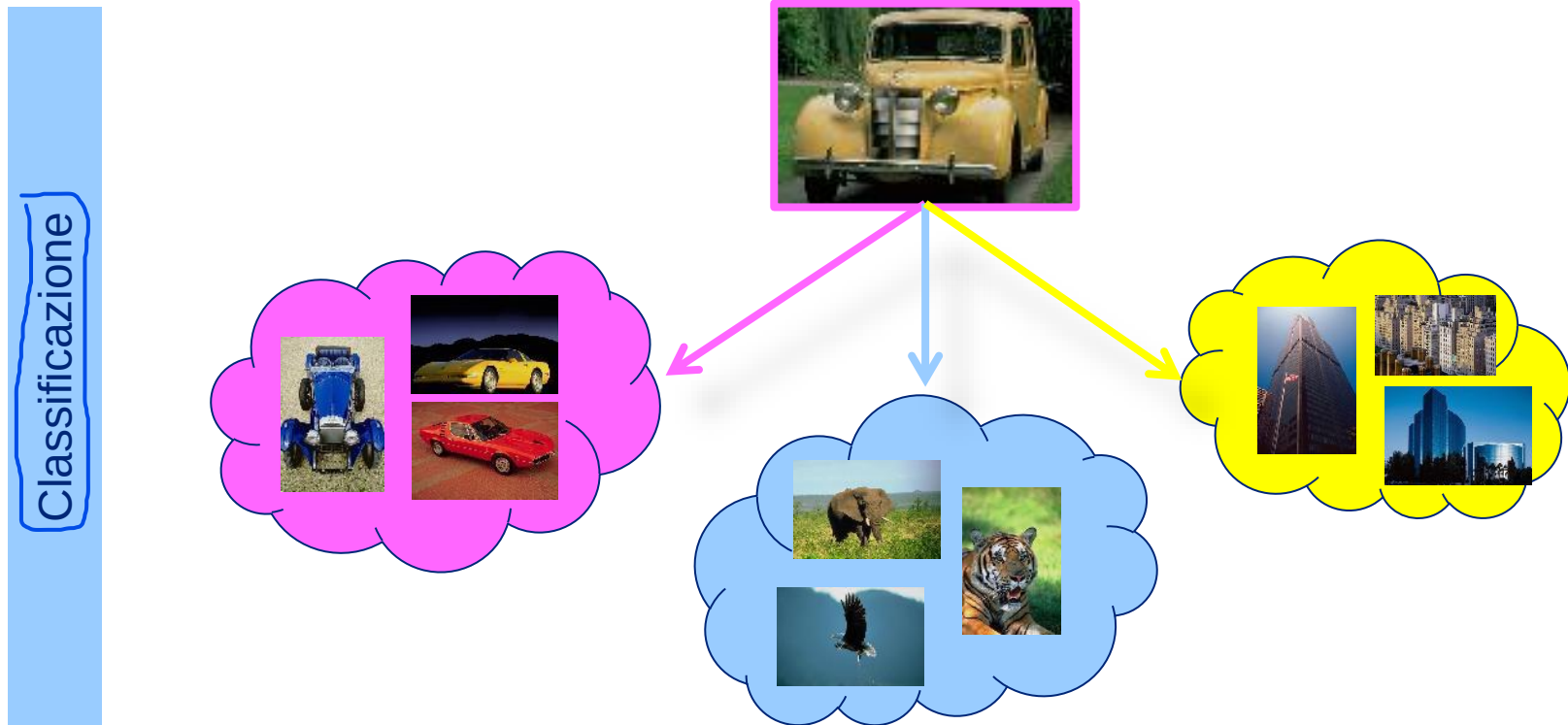


Google



Feature invariant: applicazioni (2)

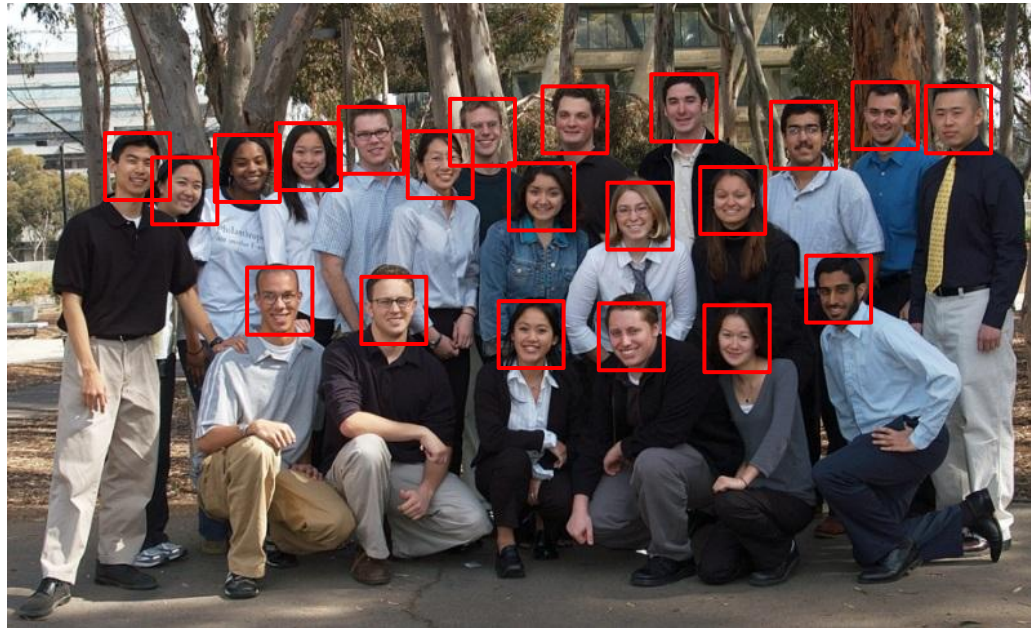
Assegnare un'immagine di input a una specifica categoria (o classe) tra un insieme di classi note.



Feature invariant: applicazioni (3)

Individuare dove si trova un particolare oggetto (o una classe di oggetti) all'interno di un'immagine complessa.

Localizzazione



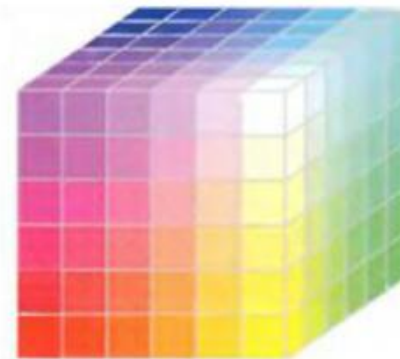
Hand-crafted features: colore



Color histograms (1)

Un **color histogram** è una rappresentazione della distribuzione dei colori in un'immagine. Nel caso di immagini digitali, l'istogramma si ottiene con i seguenti passaggi:

- Quantizzazione di ciascun canale dello spazio colore in un numero prefissato di bin (b_i);
- Assegnazione di ogni pixel dell'immagine al bin corrispondente e conteggio dei pixel;
- Normalizzazione rispetto al numero di pixel dell'immagine (invarianza rispetto alla dimensione)



$$H(i, j, k) = \#\{p: R(p) \in B_i, G(p) \in B_j, B(p) \in B_k\}$$

Color histograms (2)

- Se indichiamo con b_i il numero di bin in cui è stato discretizzato il canale i dello spazio colore (RGB o altro spazio), il numero totale di bin dell'istogramma è:

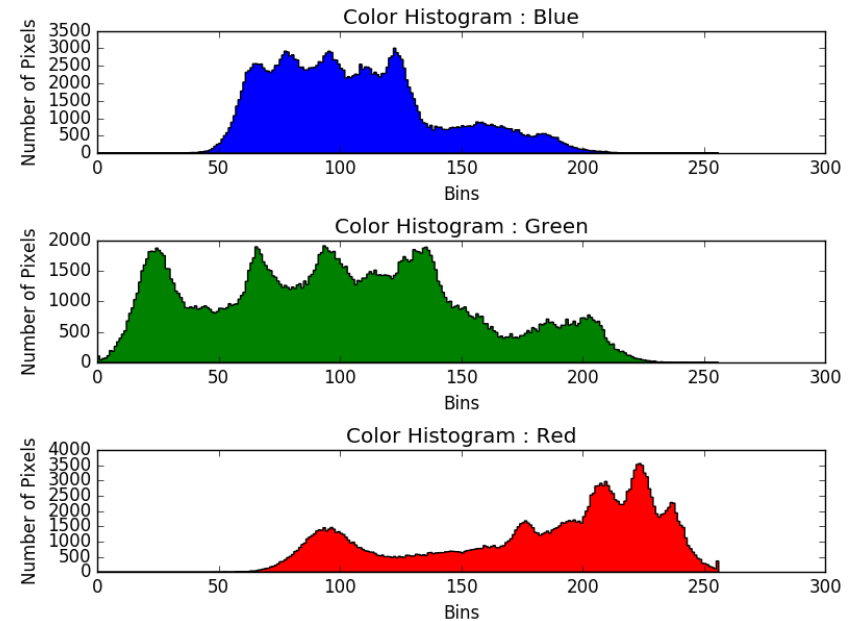
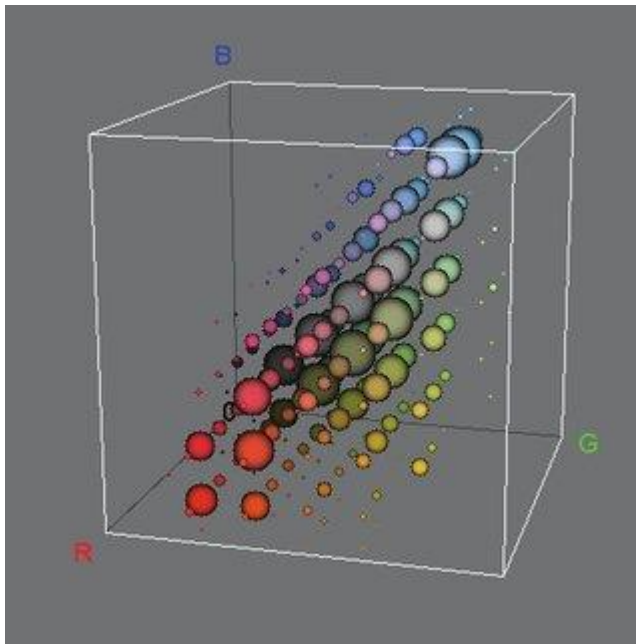
$$N = \prod_i b_i$$

- La scelta del **numero di bin** deve essere valutata attentamente perché influenza notevolmente le prestazioni sia in termine di efficacia (ciascun bin dovrebbe contenere colori effettivamente simili) sia in termini di efficienza (aumentando il numero di bin aumenta la complessità computazionale).

Color histograms (3)

- Istogramma 3D: Considera la combinazione congiunta dei canali cromatici, offrendo una rappresentazione più ricca ma con un elevato numero di bin.
- 3 Istogrammi 1D: Calcolati separatamente per ogni canale, offrendo una rappresentazione più sintetica ma con meno dettagli.

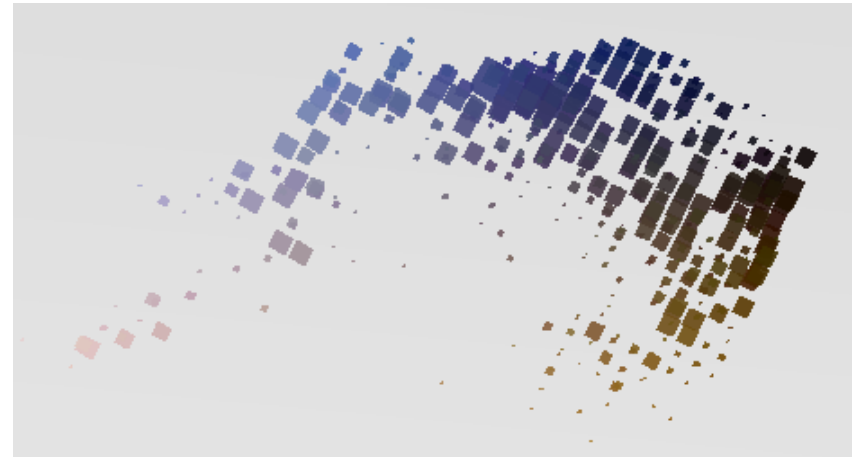
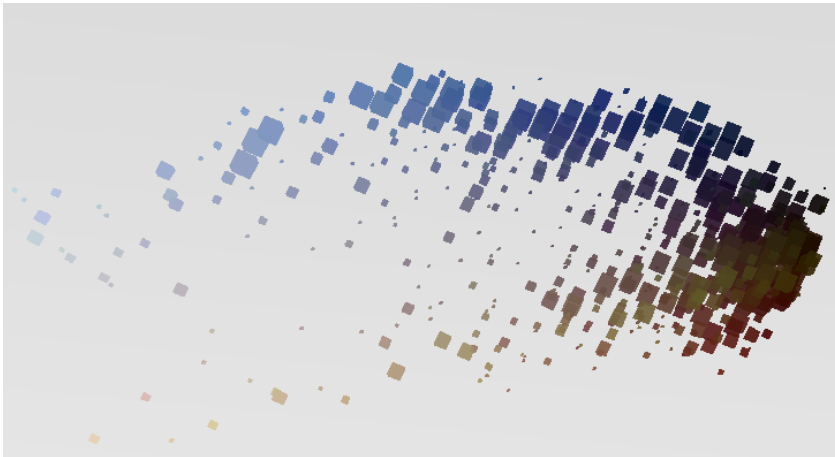
• Istogramma 3D vs. 3 istogrammi 1D



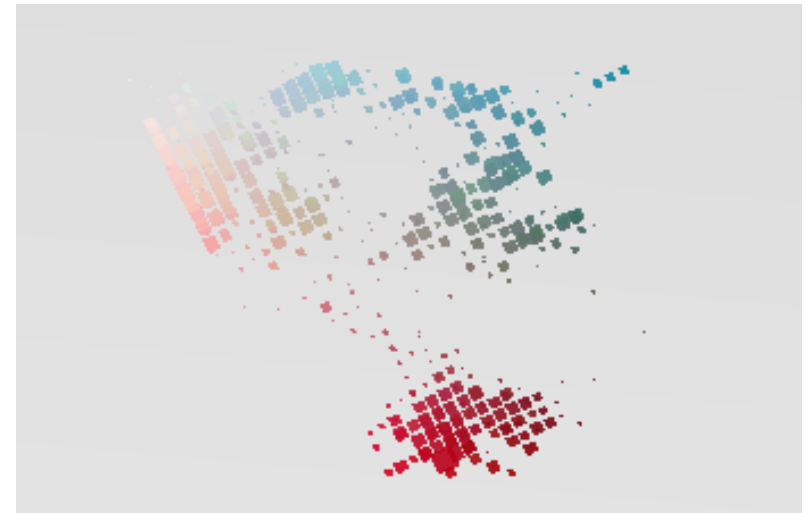
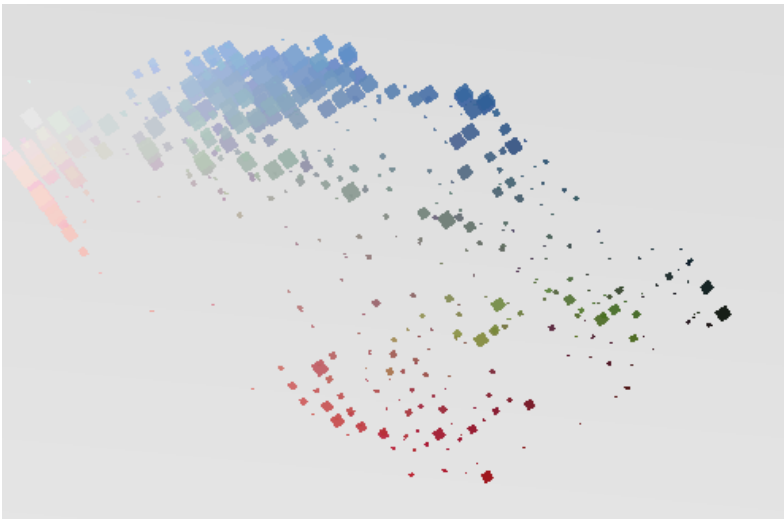
- Numero di bin elevato $N = \prod_i b_i$
- Rappresentazione più ricca

- Numero di bin ridotto $N = \sum_i b_i$
- Rappresentazione più sintetica

Color histograms: esempi (1)



Color histograms: esempi (1)



Color histograms: metriche (1)

Ai fini del confronto tra immagini sulla base degli istogrammi colore è necessario definire una metrica che ne esprima la **similarità** (o **dissimilarità**). Le metriche più usate sono:

- **Correlazione** (similarità):

$$d(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 (H_2(I) - \bar{H}_2)^2}}$$

dove $\bar{H}_k = \frac{1}{N} \sum_J H_k(J)$

- **Intersezione** (similarità):

$$d(H_1, H_2) = \sum_I \min(H_1(I), H_2(I))$$

Color moments (1)

- I **color moments** sono descrittori del colore molto utilizzati per la loro semplicità ed efficacia;
- Si basano sull'assunzione che la distribuzione dei colori in un'immagine possa essere interpretata come una **distribuzione di probabilità** caratterizzata da un certo numero di momenti che possono essere sfruttati per descrivere l'immagine stessa.

Data un'immagine I di N pixel, definiamo p_{ij} il canale i del pixel j dell'immagine; i tre momenti più utilizzati sono:

- **Media**: colore medio nell'immagine

$$E_i = \sum_{j=1}^N \frac{1}{N} p_{ij}$$

Color moments (2)

- **Deviazione standard**: radice quadrata della varianza della distribuzione del colore

$$\sigma_i = \sqrt{\left(\frac{1}{N} \sum_{j=1}^N (p_{ij} - E_i)^2 \right)}$$

- **Skewness**: misura l'asimmetria della distribuzione del colore

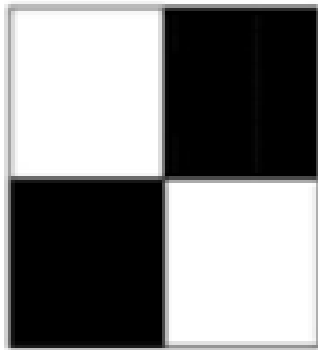
$$s_i = \sqrt[3]{\frac{1}{N} \sum_{j=1}^N (p_{ij} - E_i)^3}$$

- ✓ Il descrittore dell'immagine è costituito dai momenti calcolati per i diversi canali (RGB o altro spazio): $f = [E_R, E_G, E_B, \sigma_R, \sigma_G, \sigma_B, s_R, s_G, s_B]$.
- ✓ Il vettore di feature deve essere opportunamente **normalizzato** (diverso range di definizione dei singoli momenti)
- ✓ Per il confronto tra descrittori solitamente si usa la **distanza Euclidea**, eventualmente **pesata**.

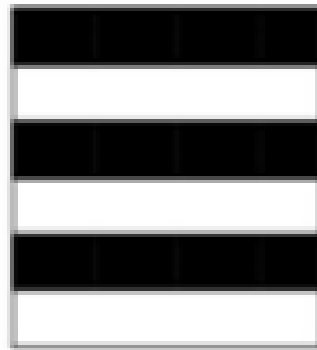
Descrittori globali o locali



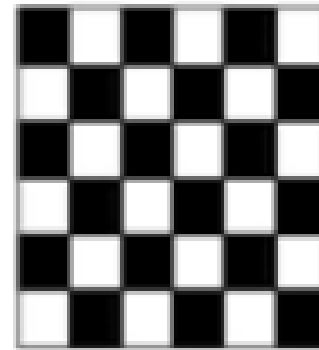
Color Histograms



A



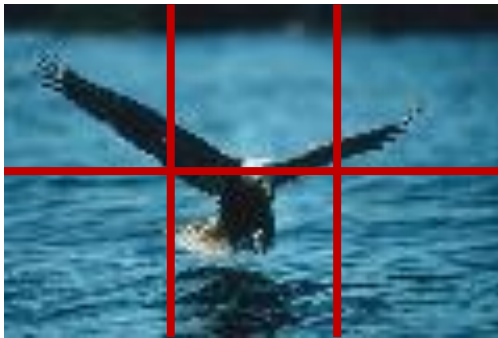
B



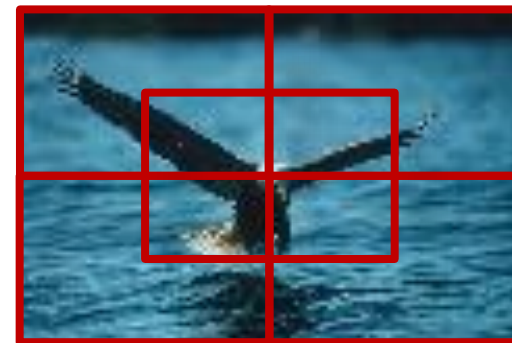
C

Descrittori globali e locali

Istogrammi e color moments, se calcolati sull'intera immagine, **non mantengono informazioni di tipo spaziale**; per arricchire il descrittore con informazioni **locali** è possibile **dividere** l'immagine in **regioni** e calcolare per ciascuna i relativi descrittori. La stessa strategia può essere usata con altri descrittori «**globali**».

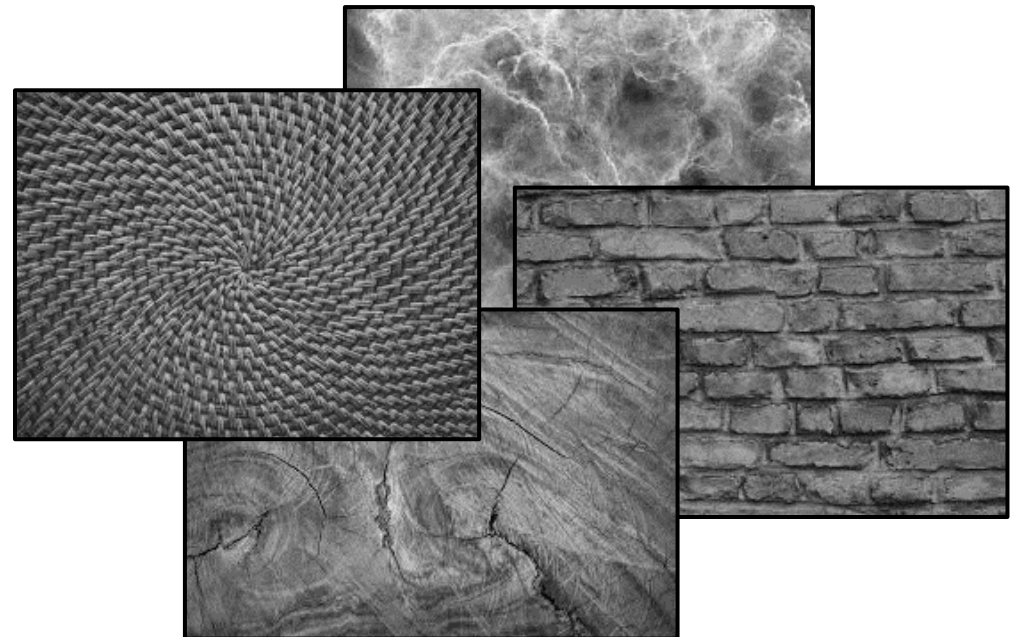


Partizionamento dell'immagine
in regioni disgiunte



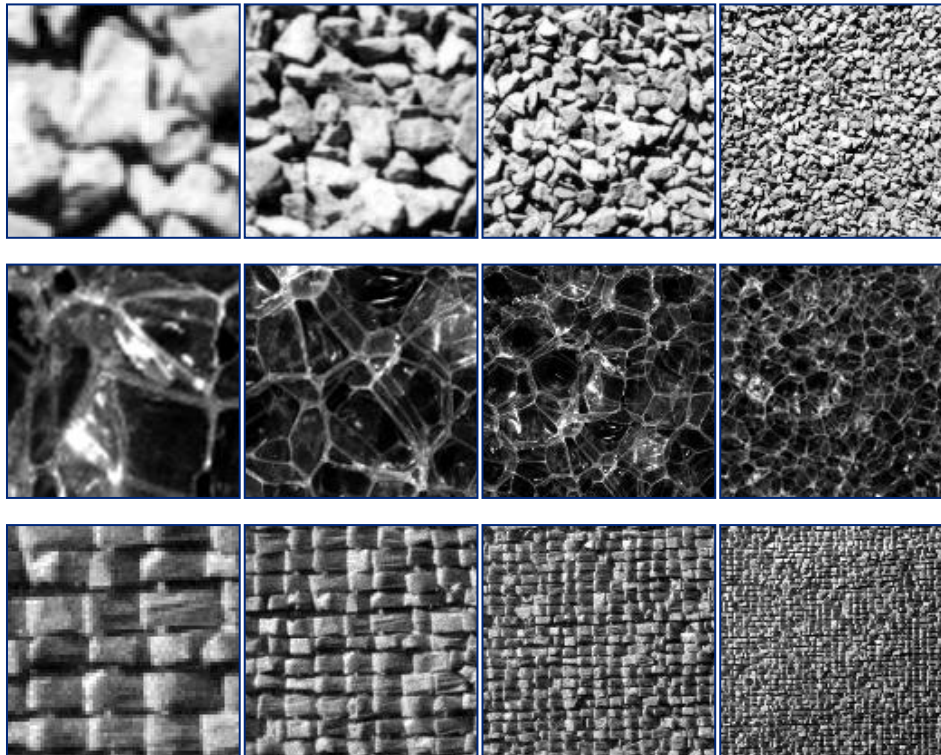
Fuzzy regions:
suddivisione in regioni
parzialmente sovrapposte

Hand-crafted features: tessitura



Tessiture (texture)

- In diverse applicazioni di PR (es: telerilevamento) i pattern presentano superfici **non omogenee** caratterizzate da **trame più o meno regolari** (dette **texture**) che possono essere sfruttate per la loro classificazione o riconoscimento.

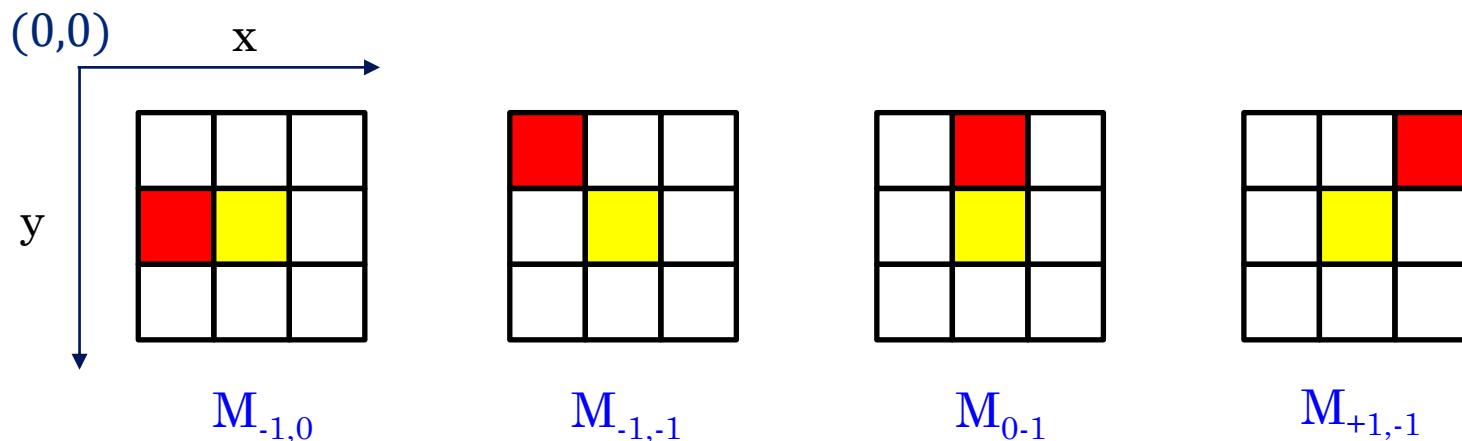


Tessiture (texture)

- Le *texture* sono generalmente caratterizzate da primitive che si ripetono secondo alcune regole che ne descrivono la trama. Dal punto di vista percettivo si distinguono tre aspetti: conformazione del granulo, direzionalità e periodicità (scala).
- Per la *discriminazione* di texture sono stati introdotti diversi metodi [Cant89] [Reed93]: Matrici di co-occorrenza, Gray-level difference, Autocorrelazione, Morfologia, Frattali, Banche di filtri.

Matrici di co-occorrenza dei livelli di grigio

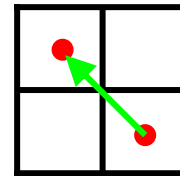
- ❑ Le matrici di **co-occorrenza** sono istogrammi bidimensionali che descrivono le co-occorrenze di coppie di livelli di grigio secondo specificate direzioni.
- ❑ Una matrice $M_{dx,dy}$ è generata da una **distanza** e una **direzione** specificate sotto forma di un **vettore spostamento** $[dx, dy]$. L'elemento $M_{dx,dy}[i, j]$ della matrice rappresenta la probabilità che il pixel $[x,y]$ abbia intensità i e il pixel $[x+dx, y+dy]$ abbia intensità j .
- ❑ Vengono generalmente calcolate le 4 matrici $M_{-1,0}$, $M_{-1,-1}$, $M_{0,-1}$, $M_{+1,-1}$ ritenute fondamentali.



Calcolo di una matrice

0	1	0	0	1	0
1	1	0	1	1	0
0	1	0	0	1	0
1	1	0	1	1	0
0	1	0	0	1	0
1	1	0	1	1	0

Immagine
(2 livelli di grigio)



$$\mathbf{d} = (-1, -1)$$

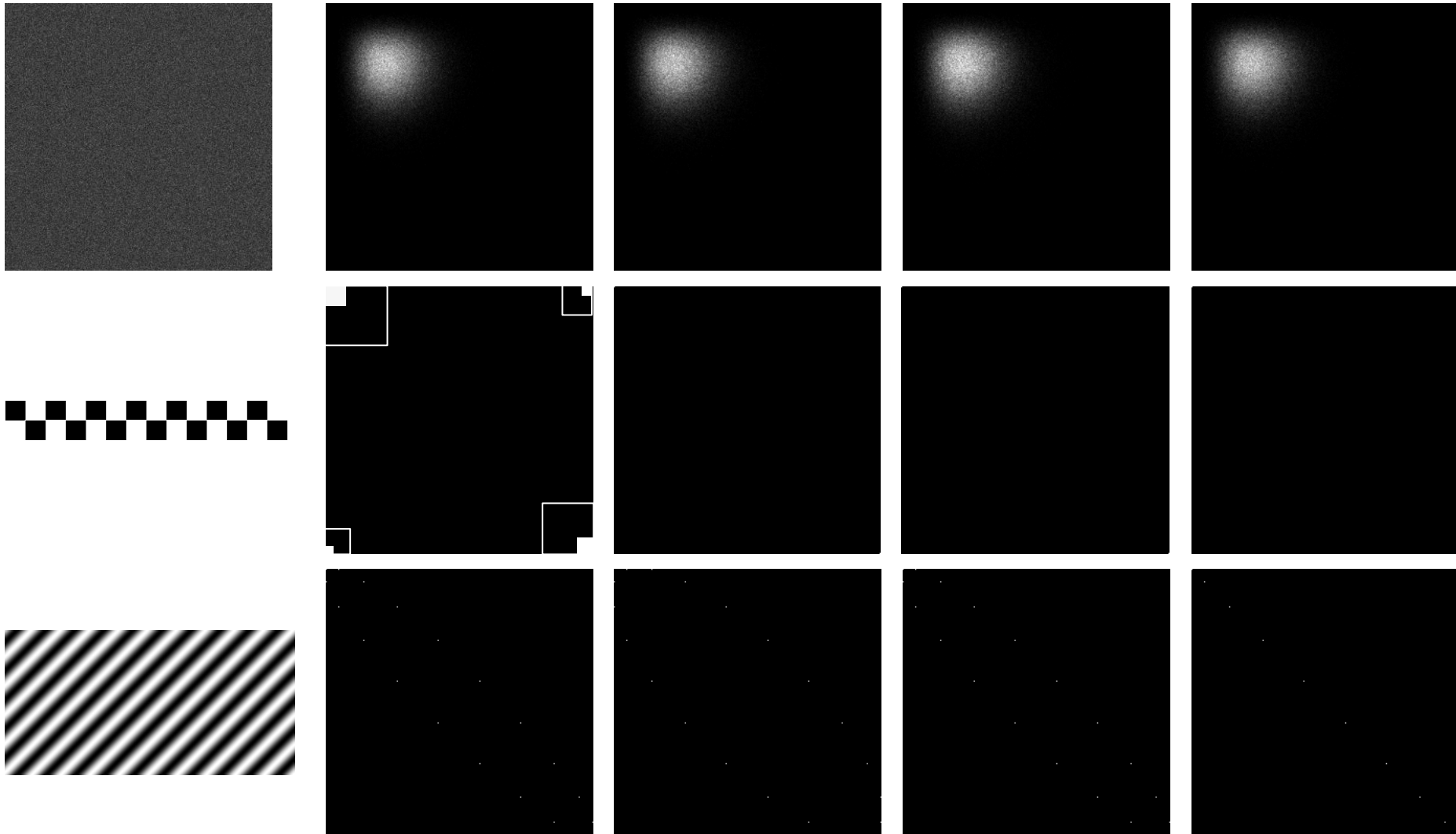
**Vettore
spostamento**

	0	1	
0	2	10	
1	9	4	

$\times 1/25$

**Matrice di
co-occorrenza**

Matrici di co-occorrenza: esempi



Matrici di co-occorrenza: misure

Dalle 4 matrici o dalla loro media (matrice non orientata) possono essere estratte numerose caratteristiche riassuntive:

Energia: $\sqrt{\sum_{i,j} M[i,j]^2}$

Raggiunge valori alti quando la distribuzione dei livelli di grigio dell'immagine è **uniforme** o periodica.

Entropia: $-\sum_{i,j} M[i,j] \log M[i,j]$

Ha valori elevati in immagini con texture non uniforme (la matrice di co-occorrenza ha molti valori bassi). Misura il grado di **"disordine"**.

Contrasto: $\sum_{i,j} (i - j)^2 M[i,j]$

Misura l'entità delle variazioni locali di luminosità.

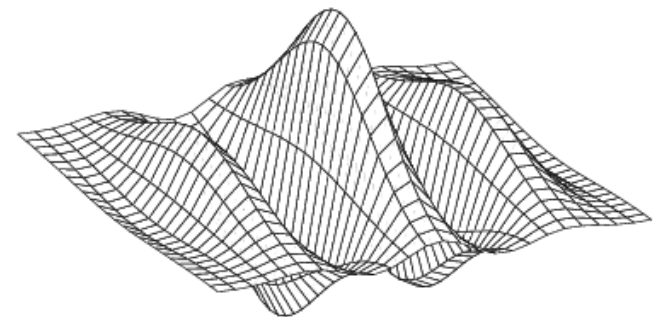
Omogeneità: $\sum_{i,j} \frac{M[i,j]}{1+|i-j|}$

Attribuisce peso massimo (1) agli elementi della diagonale e peso decrescente agli elementi extra-diagonale.

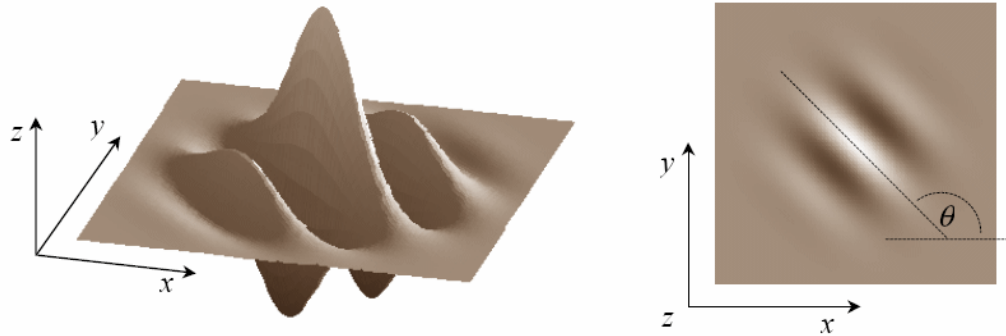
Banchi di filtri

Si tratta di eseguire la **convoluzione** dell'immagine con una serie di filtri (banco) e di estrarre dalle immagini ottenute alcuni indicatori di sintesi (media, varianza, momenti, ...).

- ❑ Un insieme di filtri molto utilizzati sono i filtri di **Gabor** [Daug88] (per i quali sono state trovate forti analogie con alcuni meccanismi del sistema visivo umano).
- ❑ Ogni **filtro** è costituito da una **funzione sinusoidale** attenuata progressivamente da una **gaussiana**. Il filtro, costituito da una parte reale e una immaginaria, è regolato da 3 parametri:
 - la **frequenza** della sinusoide ω
 - l'**orientazione** della sinusoide θ rispetto al piano x,y
 - l'**ampiezza** della gaussiana σ



Filtri di Gabor



- **Filtri di Gabor:** onda sinusoidale modulata da una distribuzione Gaussiana.
- Il filtro di Gabor simmetrico bi-dimensionale ha la seguente forma:

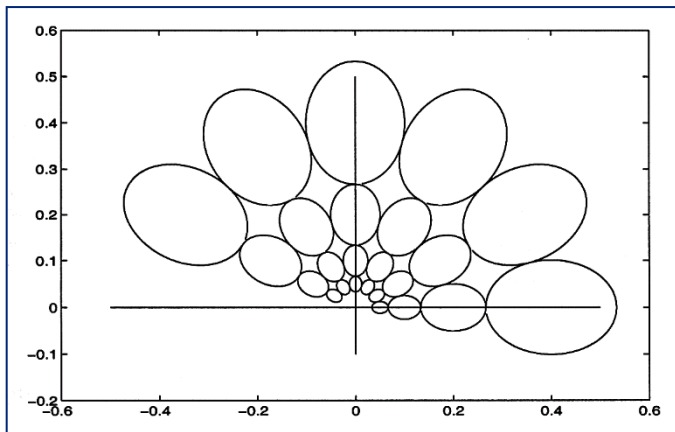
$$g(x, y : \theta, f) = \exp\left\{-\frac{1}{2}\left[\frac{x_{\theta}^2}{\sigma_x^2} + \frac{y_{\theta}^2}{\sigma_y^2}\right]\right\} \cdot \cos(2\pi f \cdot x_{\theta})$$

dove θ è l'orientazione del filtro, f è la frequenza della sinusoidale, σ_x e σ_y sono le deviazioni standard della Gaussiana negli assi x e y rispettivamente e $[x_{\theta}, y_{\theta}]$ sono le coordinate di $[x, y]$ dopo una rotazione in senso orario degli assi cartesiani di un angolo pari a $(90^\circ - \theta)$.

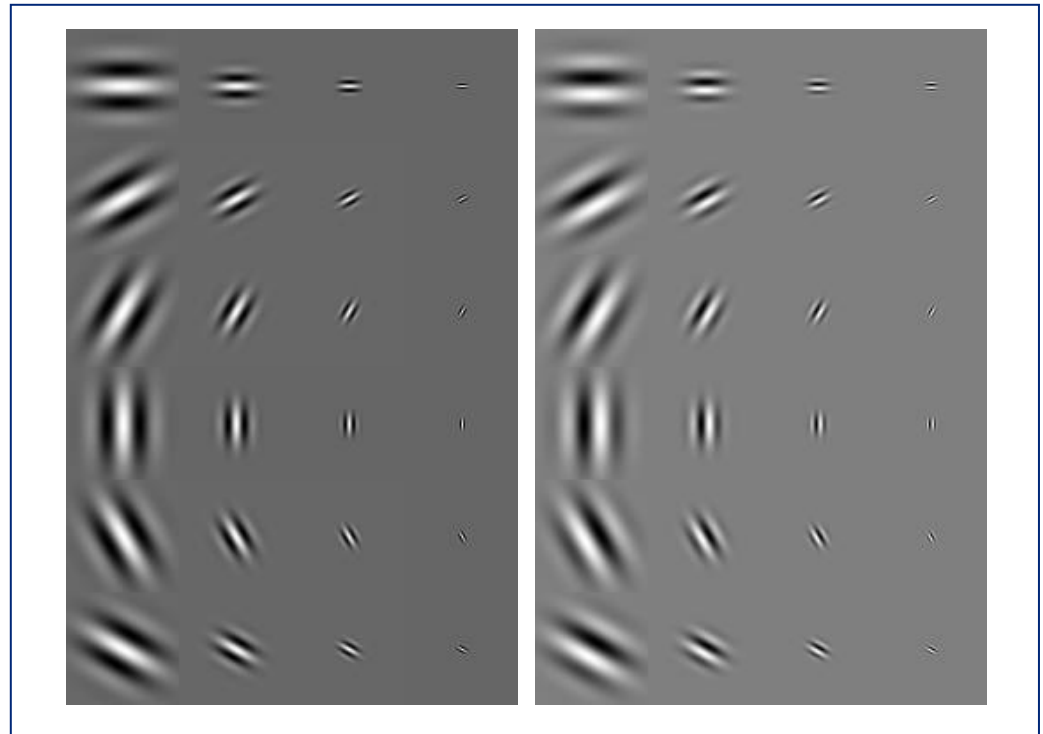
$$\begin{bmatrix} x_{\theta} \\ y_{\theta} \end{bmatrix} = \begin{bmatrix} \cos(90^\circ - \theta) & \sin(90^\circ - \theta) \\ -\sin(90^\circ - \theta) & \cos(90^\circ - \theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Filtri di Gabor

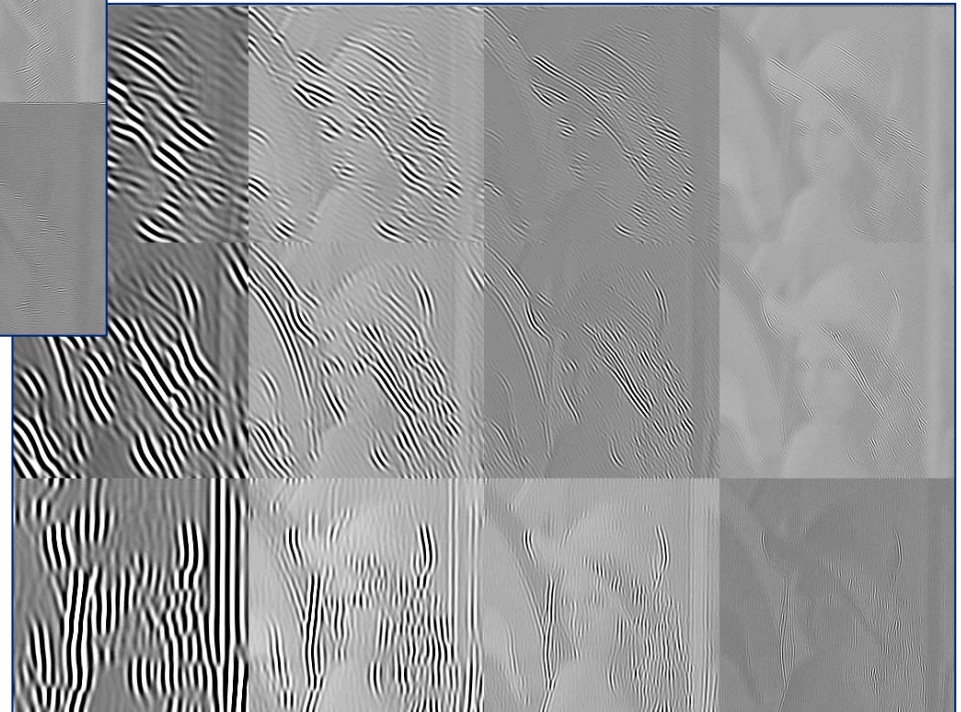
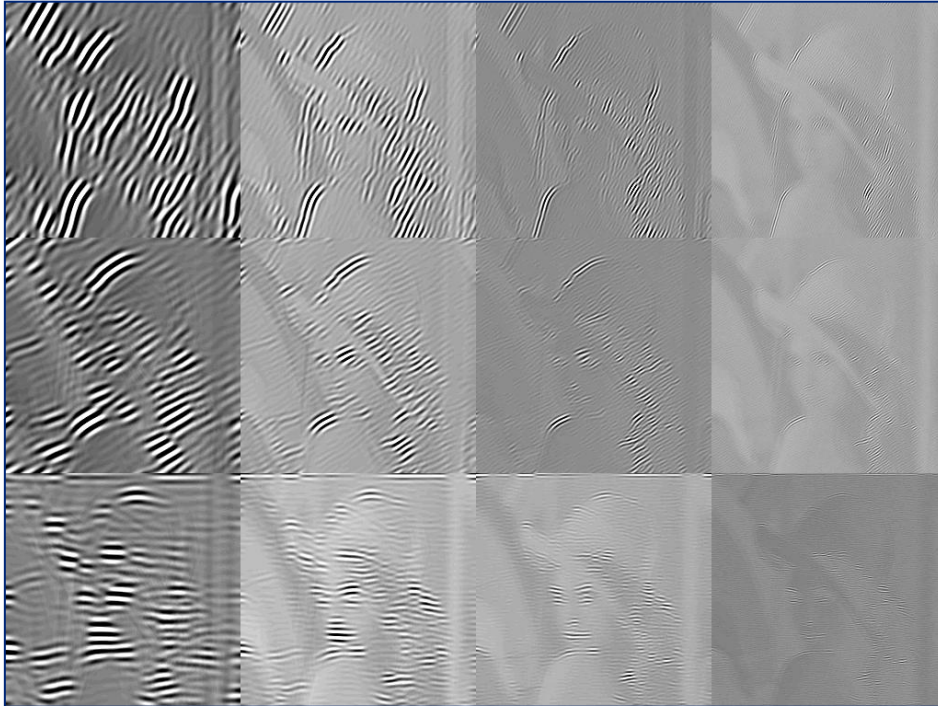
- Per la **generazione del banco** di filtri di Gabor, una volta definiti i range di interesse, si esegue una **discretizzazione** (in uno dei tanti modi possibili) dei 3 parametri fondamentali :



copertura del dominio delle
frequenze da parte dei filtri:
6 orientazioni e 4 scale

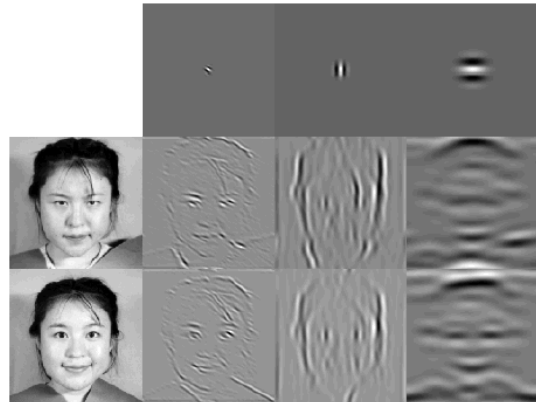


Esempio



Filtri di Gabor: estrazione di feature

Come effettuare la convoluzione?



Su tutti i pixel dell'immagine, ma il vettore di feature risultante ha dimensione molto (troppo) elevata

In corrispondenza dei nodi di una griglia uniforme sovrapposta all'immagine



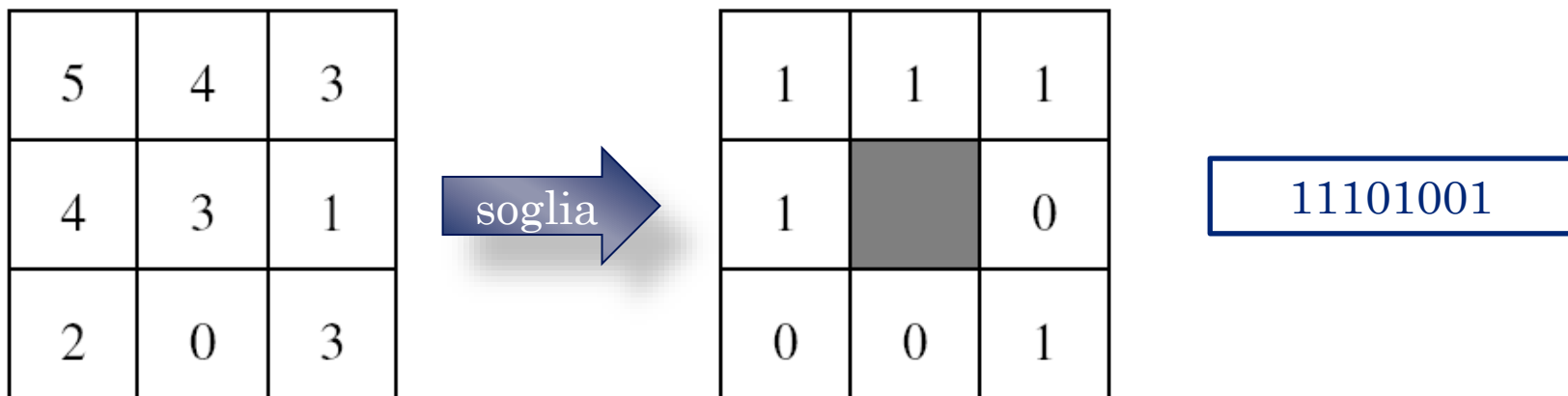
In corrispondenza dei punti salienti del volto



Il feature vector finale si ottiene concatenando la risposta (modulo o parte reale) ai diversi filtri nei punti di applicazione della convoluzione.

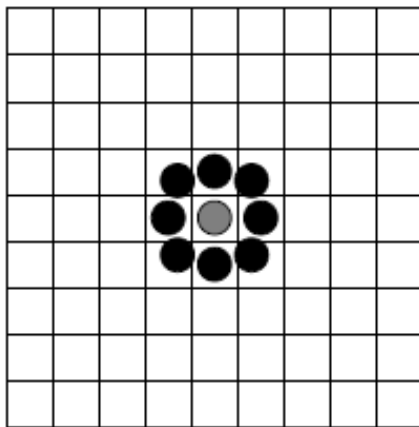
Local Binary Pattern (1)

- Il Local Binary Pattern (LBP) è un operatore introdotto da Ojala per l'analisi della **tessitura** delle immagini.
- L'operatore assegna ai pixel di un'immagine in un intorno di dimensione 3×3 un valore binario (0 o 1).
- Il valore binario è assegnato confrontando il valore del pixel con quello del pixel centrale dell'intorno:
 - Se il pixel ha un valore superiore o uguale a quello del pixel centrale gli è assegnato il valore 1, 0 in caso contrario.

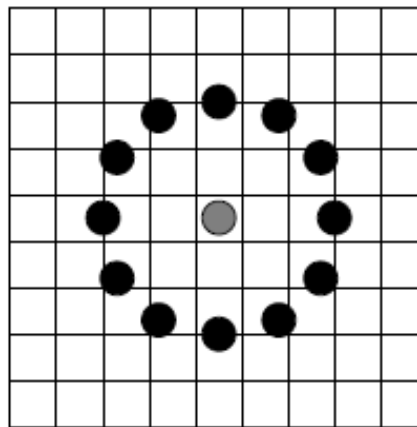


Local Binary Pattern (2)

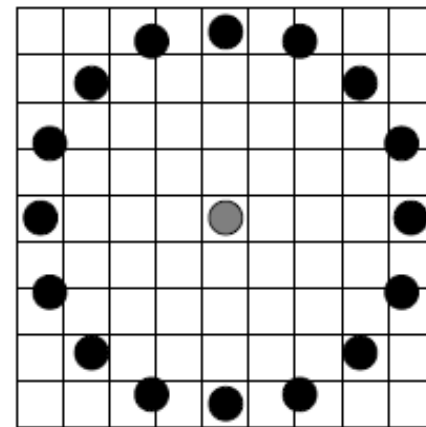
- L'operatore LBP di base è stato esteso per gestire intorni di dimensione variabile di un pixel.
- In questo caso si usa il concetto di intorno circolare di un pixel e i punti campione sono individuati tramite interpolazione.
- L'operatore in questo caso è definito da due parametri:
 - il numero di punti campione P ;
 - il raggio dell'intorno circolare R .



$P=8, R=1.0$



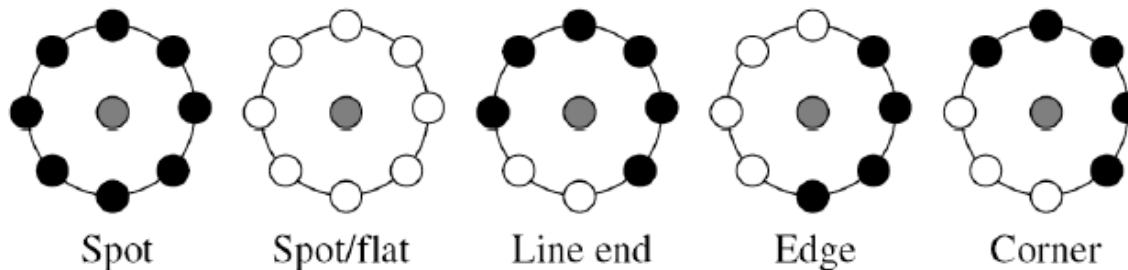
$P=12, R=2.5$



$P=16, R=4.0$

Local Binary Pattern: pattern uniformi

- I pattern binari più interessanti sono quelli *uniformi*, in quanto rappresentano le strutture locali più rilevanti (es. edge, spot, ecc).



- Un pattern è detto uniforme quando, considerato in modo circolare, contiene al massimo due transizioni 0-1 o 1-0. Ad esempio i pattern 10000011, 11110000, 00000000 sono uniformi.
- Considerare solo i pattern uniformi permette di risparmiare memoria: i pattern totali sono 2^P , mentre i pattern uniformi sono solo $P \cdot (P-1) + 2$.



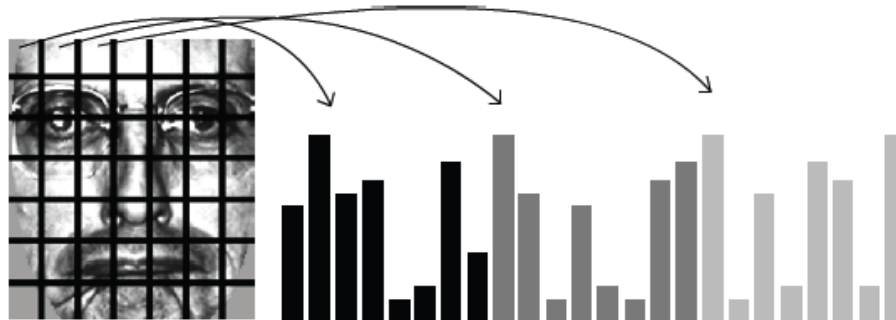
*Pixel con pattern
uniforme*



*Pixel con pattern
non uniforme*

Local Binary Pattern: feature vector

- Il vettore di feature associato a un'immagine è un **istogramma** (eventualmente normalizzato) calcolato come segue:
 - L'immagine è partizionata in k^2 sottofinestre.
 - Per ogni sottofinestra è costruito un istogramma in cui ciascun bin è associato a uno specifico pattern; i bin sono in totale $P \cdot (P-1) + 3$: $P \cdot (P-1)$ bin per i pattern con 2 transizioni, 2 bin per i pattern con 0 transizioni, 1 bin per i pattern non uniformi.
 - Il vettore di feature si ottiene concatenando gli istogrammi calcolati per tutte le sottofinestre.



Binarized Statistical Image Features (1)

<http://www.ee.oulu.fi/~jkannala/bsif/bsif.html>

- Metodo proposto per l'analisi di tessiture; utilizzato per varie applicazioni nell'ambito del riconoscimento del volto (es. PAD);
- Il metodo Binarized Statistical Image Features (BSIF), analogamente ad LBP, rappresenta un'immagine come un **istogramma di occorrenze di pattern binari**;
- Ciascun pattern binario codifica il contenuto visivo di una «patch» dell'immagine;
- I singoli pattern binari sono calcolati **binarizzando la risposta a un insieme di filtri lineari**;
- I filtri utilizzati non sono pre-definiti ma **appresi** attraverso un opportuno addestramento (tipicamente a partire da immagini naturali);
- I filtri sono stimati in fase di training, con l'obiettivo di massimizzare l'indipendenza statistica delle stringhe binarie risultanti dalla loro applicazione (tramite ICA).

Binarized Statistical Image Features (2)



Patch X ($l \times l$)

$$s_i = \sum_{u,v} W_i(u,v) \cdot X(u,v)$$

i -esimo filtro lineare

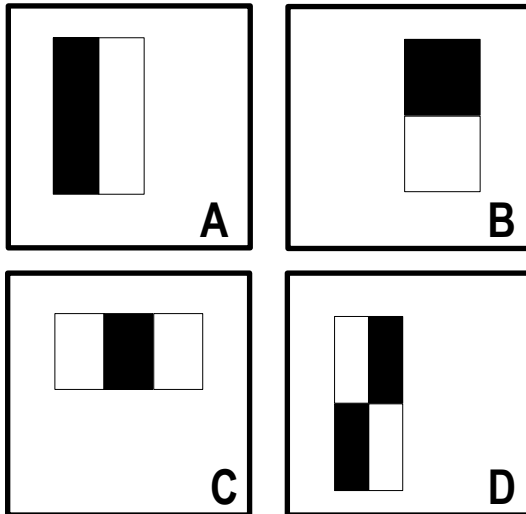


- Per ciascuna patch nell'immagine si determina la risposta s_i a ciascun filtro W_i ;
- Il valore s_i viene poi binarizzato:

$$b_i = \begin{cases} 1 & \text{se } s_i > 0 \\ 0 & \text{altrimenti} \end{cases}$$

- La stringa binaria **b** associata alla patch si ottiene concatenando i valori di b_i (ottenuti applicando il filtro W_i).
- L'intera immagine è rappresentata dall'istogramma delle occorrenze delle possibili stringhe binarie **b**.

Haar-like features



- La texture che caratterizza un'immagine può essere codificata applicando all'immagine stessa delle maschere binarie che evidenziano particolari variazioni di intensità luminosa.
- Ogni valore del vettore di feature è ottenuto posizionando una maschera in una sottoregione dell'immagine.
- Le feature sono applicate modificandone la dimensione, la forma, e la posizione nell'immagine.

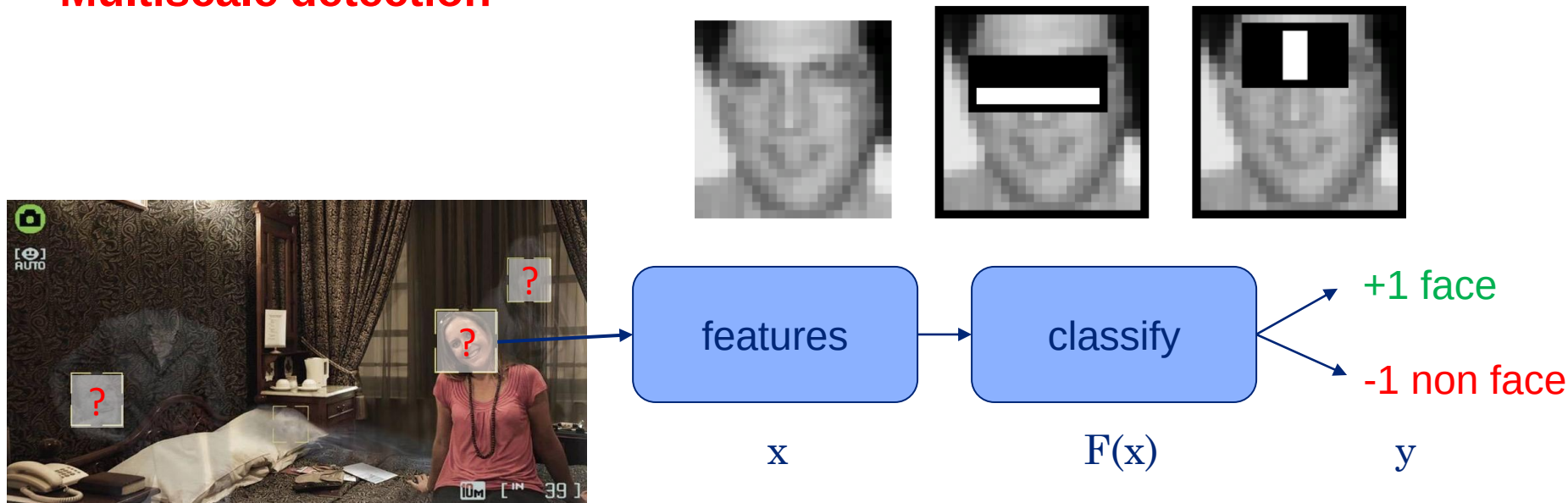
Usando una sottofinestra di dimensione 24×24 , con tutte le possibili combinazioni di posizione (orizzontale e verticale) e scala, si ottengono in totale 49396 feature.

Queste feature sono utilizzate poiché:

- sono efficaci per molte applicazioni (es. localizzazione del volto);
- possono essere calcolate in modo efficiente usando l'*immagine integrale*.

Un'applicazione: il localizzatore di Viola e Jones

- Estrazione e valutazione di **Haar-like feature**
- Classificazione mediante **boosting** (costruzione di un classificatore robusto come combinazione di molteplici classificatori semplici)
- **Multiscale detection**



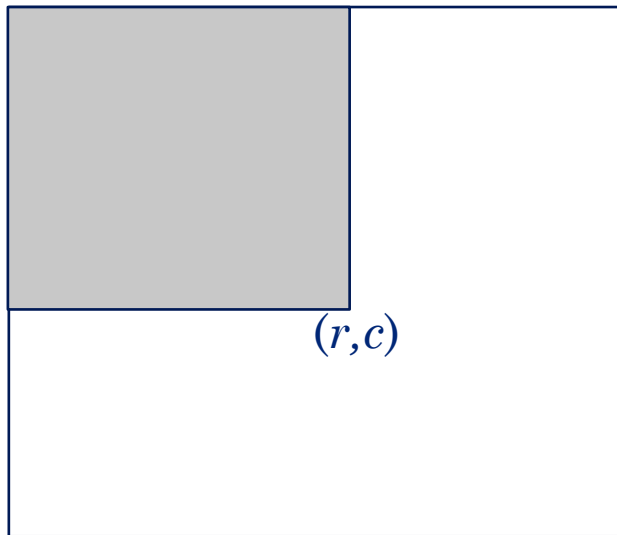
la localizzazione è effettuata facendo scorrere una finestra di ricerca (le cui dimensioni possono variare) sull'immagine, estraendo le feature presenti nella finestra e classificando la finestra come volto o non volto.

L'immagine integrale

L'**immagine integrale** in posizione (r,c) è la somma del valore dei pixel sopra e a sinistra di (r,c) :

$$II(r,c) = \sum_{r' \leq r, c' \leq c} I(r',c')$$

L'immagine integrale può essere calcolata con **una sola scansione** dell'immagine originale usando le seguenti formule ricorsive:

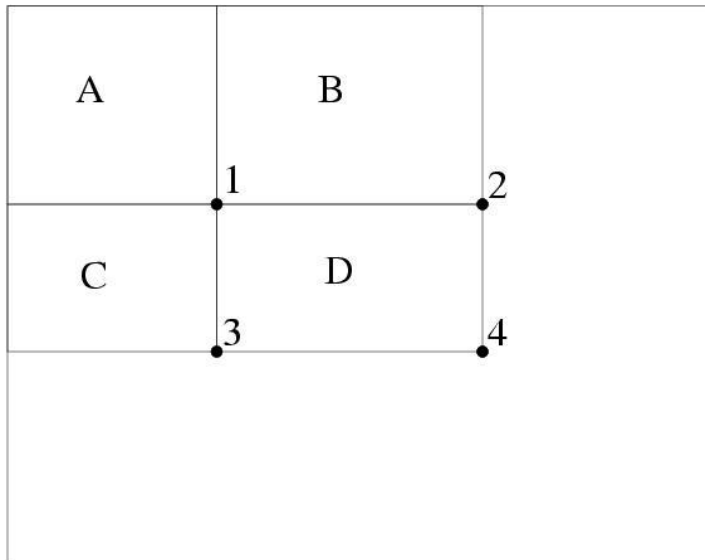


$$S(r,c) = S(r,c-1) + I(r,c)$$

$$II(r,c) = II(r-1,c) + S(r,c)$$

dove $I(r,c)$ è il valore del pixel nell'immagine originale, $S(r,c)$ è la somma cumulativa per righe, $S(r,-1)=0$ e $II(-1,c)=0$.

Calcolo efficiente di feature rettangolari



Usando l'immagine integrale, è possibile calcolare la somma dei pixel in qualsiasi rettangolo.

Ad esempio la somma all'interno del rettangolo D è data da:

$$I(4) + I(1) - I(2) - I(3)$$

Le feature caratterizzate da 2, 3 o 4 rettangoli possono essere calcolati usando rispettivamente 6, 8 e 9 valori di riferimento.